

Siw Books

Appunti progetto sistemi informativi su web

Anno Accademico: 2024/25

Mattia Cosimi

*Dipartimento di Ingegneria Civile, Informatica e delle Tecnologie Aeronautiche
Università degli Studi “Roma Tre”*

Indice

1	Come svolgere il progetto	1
1.1	Inizio: setup del progetto	1
1.1.1	Repository	1
1.1.2	Service	1
1.1.3	Controller	1
2	Struttura della HomePage	1
2.1	Introduzione	1
2.2	Spring Security	1
3	Implementazione delle pagine	2
3.1	Login	2
3.2	Signup	2
3.3	Pagina dei libri	2
3.4	Pagina degli autori	2
3.5	Pagina del libro singolo	3
4	ADMIN Section	3
4.1	Aggiunta di un libro	3
4.2	Aggiunta di un autore	3
5	Appunti	3
5.1	Varie	3
6	TicketLuna Appunti	3
6.1	Registrazione e login	3
6.2	Home Page	3
6.2.1	CountTicketsByOwner (Biglietti totali)	4
6.2.2	CalculateTotalRevenue (Entrate totali)	4
6.2.3	MostPopularPark (LunaPark più popolare)	4
6.2.4	GetMostPopularParkTicketCount (Num ticket venduti lunaPark popolare)	4
6.2.5	CalculateMonthlyGrowth	4
7	Soft-Delete	4
7.1	Come funziona?	4
8	Come passare gli attributi	5
8.1	Model	5
8.2	RequestParam	5
8.3	ModelAttribute	5
8.4	PathVariable	5

1 Come svolgere il progetto

1.1 Inizio: setup del progetto

Inizio creando i pacchetti e le cartelle necessarie, scrivo poi le classi e i loro vincoli, scrivo classi service e repository. Per quanto riguarda l'aggiunta delle immagini, dovendole salvare in locale utilizzo il tipo BLOB. Dovrò fare delle classi apposite per mettere più immagini per libro (Lo implementerò prossimamente). Devo gestire le relazioni tra le classi, decido di utilizzare una relazione OneToOne bidirezionale tra User e Review dato che lo user può fare più recensioni e la recensione è scritta da uno user. Essendo un'associazione bidirezionale è necessario che ci sia reciprocità dei riferimenti per farlo dalla parte di OneToMany inserisco mappedBy. Finite le associazioni inserisco i messaggi da mostrare quando i vincoli non vengono rispettati e implemento repository e service.

1.1.1 Repository

Il package repository contiene le classi responsabili dell'accesso ai dati, cioè della comunicazione con il database. Estende CRUD repository che mi permette di avere dei metodi già salvati, o creare dei metodi personalizzati che JPA implementa automaticamente. Il repository è quindi lo strato per la persistenza dei dati.

1.1.2 Service

Il package service invece, contiene la logica applicativa, ovvero quello che succede nel sistema al verificarsi di un'azione. Ecco perchè **controller** gestisce le richieste HTTP → **service** la logica applicativa → **repository** le richieste al database.

1.1.3 Controller

Il controller si occupa di gestire le richieste HTTP, è un punto di interazione tra l'utente (via browser) e l'applicazione. Quindi il controller riceve una richiesta, la inoltra al service e restituisce una risposta. Generalmente nei metodi del controller troviamo model il quale ci serve per passare i dati al template html.

2 Struttura della HomePage

2.1 Introduzione

Inizio a strutturare l'home page per capire cosa voglio poi andare a implementare. Bene una volta capito che direzione voglio prendere, è importante andare a implementare la logica di spring security.

2.2 Spring Security

Inizio a creare la classe credentials, la quale avrà il compito di memorizzare username e password criptate degli utenti. La strategia utilizzata sarà quella di utilizzare i ruoli per capire quando siamo

loggati da admin e quando da utenti. Sui link riservati agli owner vado a specificare il ruolo che si deve avere per accedervi. Il logout funziona solo se si è autenticati e login e signup solo se non si è autenticati. Implemento il **global controller**: Serve a rendere disponibili in automatico alcune informazioni sugli utenti loggati a tutte le pagine del sito, senza doverle aggiungere manualmente in ogni controller. Creo un pacchetto chiamato authentication il quale conterrà **authConfiguration** che serve a configurare la sicurezza della mia applicazione e quindi pagine come il login, chi può fare cosa, e come vengono gestite le pagine. L' **authenticationController** gestirà come vengono autenticati e registrati gli utenti nella mia applicazione web. Aggiungo poi **CredentialsService** che gestirà le operazioni sulla gestione delle credenziali utente.

3 Implementazione delle pagine

3.1 Login

Implementata la home page inizio a implementare le altre pagine a partire dalla pagina di login. La struttura è uguale a quella di ticket luna a eccezione per il css. Per quanto riguarda la logica dietro vi è un "pop up" che appare solo dopo essersi registrati correttamente dato che sign-up ha come post mapping login, ha un attributo error in caso vi siano errori nel login, il form ha un post mapping in modo tale da inviare le informazioni inserite nel form.

3.2 Signup

Il signup (stessa struttura di ticket luna) ha anche lui un form con un post che sostanzialmente, controlla se le credenziali sono già esistenti, se non esistono, si controlla se ci sono errori nell'inserimento e se non ci sono errori viene salvato lo user e le credenziali.

3.3 Pagina dei libri

Finito il signup inizio a strutturare la pagina dei libri. La pagina dei libri ovviamente avrà una struttura molto simile a quella degli autori, il controller quindi cosa farà invocherà un metodo del service il quale ci fornisce tutti i libri (un iterabile) da mandare poi alla pagina che scandirà con un for each. Il repository ottiene l'iterabile con un metodo crud, findall. Implemento il for each e capisco un pò come strutturare la pagina, vorrei impostare la ricerca di un libro, e sotto la ricerca i libri mostrati. Per fare questo e capire come possa venire cerco di fare un import sql. Faccio l'import sql che però non mi permette di aggiungere le immagini e quindi inizio a costruire il form che permetterà all'admin di aggiungere un nuovo libro.

3.4 Pagina degli autori

Questa pagina sarà pressochè uguale a quella dei libri.

3.5 Pagina del libro singolo

Inizio cambiando il bottone view details in modo tale che mi riporti a una pagina dedicata solo a quel libro.

4 ADMIN Section

4.1 Aggiunta di un libro

Ovviamente per quanto riguarda il lato controller, avremo un get mapping il quale ci tornerà un oggetto libro da riempire e la pagina. Per aggiungere il libro, mi servono gli autori, allora faccio prima la pagina dove vengono mostrati gli autori e poi l'aggiunta di un nuovo autore. Dopo aver fatto con un for each il display degli autori, voglio renderli selezionabili, per farlo, aggiungo una query al repository che inserisce la relazione tra l'autore e il libro. Come gestisco il lato controller: prendo la lista degli autori da aggiungere, e aggiungo per ogni autore una riga alla relazione tra autore e libro. Nell'aggiunta di un libro ho inserito la possibilità di mettere gli autori che vengono mostrati tramite una checkbox invisibile, e con una gestione di javascript si crea l'illusione di selezionare e deselectare.

4.2 Aggiunta di un autore

Questa pagina è analoga a quella del libro. Come funziona il Post mapping, se ci sono degli errori, si ritorna al form, se non ci sono errori, si salva l'autore e si torna sulla pagina degli autori. Ho dovuto fare dei metodi nel controller per la gestione delle immagini e una classe a parte.

5 Appunti

5.1 Varie

Dopo aver fatto queste cose ho preso il via e non ho segnato più nulla. @GetMapping serve per leggere dati, mai per modificarli. @PostMapping per modificarli.

6 TicketLuna Appunti

6.1 Registrazione e login

Per quanto riguarda registrazione e login, le cose sono uguali a siw book.

6.2 Home Page

L'utente e l'admin hanno due home page differenti.

6.2.1 CountTicketsByOwner (Biglietti totali)

Questo metodo di ticket service, si appoggia a un metodo del repository, il quale somma tutte le quantità di ticket dove l'owner è quell'owner e viene poi controllato che il lunaPark relativo non sia stato soft cancellato. Viene aggiunto coalesce per fare in modo che se venga restituito null quest'ultimo venga sostituito con 0. **RelatedTo** è il lunaPark per il ticket.

6.2.2 CalculateTotalRevenue (Entrate totali)

Nel service, prima vengono presi tutti i biglietti di quell'owner con il lunaPark visibile (grazie a una query del repository) e poi usa lo Stream API per calcolare la somma dei prezzi di vendita di una lista ignorando i null. Crea uno stream, trasforma ogni ticket in un double, controlla se un prezzo di vendita è null, se non è null lo usa, se è null lo sostituisce con 0.0.

6.2.3 MostPopularPark (LunaPark più popolare)

Questo metodo del service, si appoggia a un metodo del repository, il quale restituisce la lista di id lunapark e somma di biglietti venduti e nel service si prende il primo lunaPark della lista e lo restituisce.

6.2.4 GetMostPopularParkTicketCount (Num ticket venduti lunaPark popolare)

Questo si appoggia alla stessa lista di prima, stavolta però ritorna il secondo elemento del primo elemento (in modo tale da avere il numero di biglietti venduti).

6.2.5 CalculateMonthlyGrowth

Si prende tutti i biglietti del mese attuale, tutti i biglietti del mese precedente e calcola qual'è la percentuale di biglietti venduti in più.

7 Soft-Delete

7.1 Come funziona?

Gli elementi che hanno questa funzionalità e quindi **lunaPark** e **PerformanceDescription** hanno un attributo settato di default a true che ne rappresenta la visibilità. Quando lo voglio cancellare quindi, chiamo un metodo sul service che si chiama softDeleteNome il quale prende come parametro l'entità, gli setta la visibilità a false e lo salva. Questo è fatto in modo tale che se l'owner decide di cancellare un lunaPark o un attrazione, quest'ultima rimanga comunque all'interno dei biglietti già acquistati.

8 Come passare gli attributi

8.1 Model

Con il model all'interno dei controller spring riesco a passare degli oggetti al browser.

8.2 RequestParam

Con RequestParam chiedo un attributo dal browser al controller spring il nome è il name specificato in html.

8.3 ModelAttribute

Con ModelAttribute mi riesco a passare ad esempio lo user corrente. In che modo: specificando sopra il metodo modelAttribute, rendo il parametro disponibile al browser e ai metodi dei controller. Infatti quello che succede è che il parametro viene inserito nel model.

8.4 PathVariable

PathVariable si usa quando vi è un'oggetto che si passa all'interno dell'url. Principalmente si passano gli Id.